1. 试解释在十字路口产生的交通死锁现象中包括了死锁产生的4个必要条件，并给出避免产生交通死锁的简要规则。

① 互斥．一个位置只有一辆车
② 不剥夺．不能挪走车．
③ 请求和保持．一辆车有一个位置且保持前进
④ 循环等待．每辆车等待前方资源

交警让车倒退（破坏不剥夺条件）
车辆还进入路口可通过（破坏请求与保持条件）静态分配
直行优先，左转其次右转（破坏循环等待条件）
顺序资源分配

2. 设三个进程共享四个资源，每次资源分配与请求只能以一个单位进行。已知每一进程最多需要2个单位的资源，那么系统中会发生死锁现象吗？试说明原因。

不会，

| No | Max | Allocation | Need |
|---|---|---|---|
| 1 | 2 | 1 | 1 |
| 2 | 2 | 1 | 1 |
| 3 | 2 | 1 | 1 |

此时分配给谁都安全

3. 系统在某一时刻的状态如表1所示

表1 系统状态

| | Allocation | | | | Max | | | | Available | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | A | B | C | D | A | B | C | D |
| $P_0$ | 0 | 0 | 1 | 2 | 0 | 0 | 1 | 2 | 1 | 5 | 2 | 0 |
| $P_1$ | 1 | ~~0~~ 4 | ~~0~~ 2 | 0 | 1 | 7 | 5 | 0 | | | | |
| $P_2$ | 1 | 3 | 5 | 4 | 2 | 3 | 5 | 6 | | | | |
| $P_3$ | 0 | 6 | 3 | 2 | 0 | 6 | 5 | 2 | | | | |
| $P_4$ | 0 | 0 | 1 | 4 | 0 | 6 | 5 | 6 | | | | |

使用银行家算法回答下列问题：

(1) 请给出*Need*矩阵。

(2) 当前系统是否处于安全状态？

(3) 如果从进程$P_1$发来一个请求(0,4,2,0)，这个请求能否立刻满足？

(1). Need = Max - Allocation =

| | A | B | C | D |
|---|---|---|---|---|
| P0 | 0 | 0 | 0 | 0 |
| P1 | 0 | 7 3 | 5 3 | 0 |
| P2 | 1 | 0 | 0 | 2 |
| P3 | 0 | 0 | 2 | 0 |
| P4 | 0 | 6 | 4 | 2 |

(2). 使用银行家算法检测

① P0 释放 A=(1,5,3,2)

② 给P2 尝试 A=(2,8,8,6)

③ 给P1

④ 给P3　　存在安全序列 <P0,P2,P1,P3,P4>，处于安全状态

⑤ 给P4

(3). 尝试分配. A=(1,1,0,0)

①P0 释放. A=(1,1,1,2)

②给P2 释放 A=(2,4,6,6)

③ 给P1

④ 给P3

⑤ 给P4.

存在安全序列 <P0,P2,P1,P3,P4>. 安全

**（2）当前系统是否处于安全状态？**

初始化矢量 **Work=Available(1，5，2，0)**

系统安全性分析：

| 进程 | Work | | | | Need | | | | Allocation | | | | Available | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | A | B | C | D | A | B | C | D | A | B | C | D |
| P0 | 1 | 5 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 1 | 5 | 3 | 2 |
| P3 | 1 | 5 | 3 | 2 | 0 | 0 | 2 | 0 | 0 | 6 | 3 | 2 | 1 | 11 | 6 | 4 |
| P2 | 1 | 11 | 6 | 4 | 1 | 0 | 0 | 2 | 1 | 3 | 5 | 4 | 2 | 14 | 11 | 8 |
| P1 | 2 | 14 | 11 | 8 | 0 | 7 | 5 | 0 | 1 | 0 | 0 | 0 | 3 | 14 | 11 | 8 |
| P4 | 3 | 14 | 11 | 8 | 0 | 6 | 4 | 2 | 0 | 0 | 1 | 4 | 3 | 14 | 12 | 12 |

CSDN @风云诀

因为存在一个安全序列<P0、P3、P2、P1、P4>，所以系统处于安全状态

**（3）如果从进程 P1发来一个请求(0,4,2,0)，这个请求能否立刻满足？**

**$Request_1(0,4,2,0) < Need_1(0,7,5,0)$**
**$Request_1(0,4,2,0) < Available(1,5,2,0)$**

假设先试着满足进程 P1 点这个请求，则 Available 变为(1,1,0,0)

系统状态变化为：

| 进程 | Max | | | | Allocation | | | | Need | | | | Available | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | A | B | C | D | A | B | C | D | A | B | C | D |
| P0 | 0 | 0 | 1 | 2 | 0 | 0 | 1 | 2 | 0 | 0 | 0 | 0 | | | | |
| P1 | 1 | 7 | 5 | 0 | 1 | 0 | 0 | 0 | 0 | 7 | 5 | 0 | | | | |
| P2 | 2 | 3 | 5 | 6 | 1 | 3 | 5 | 4 | 1 | 0 | 0 | 2 | 1 | 1 | 0 | 0 |
| P3 | 0 | 6 | 5 | 2 | 0 | 6 | 3 | 2 | 0 | 0 | 2 | 0 | | | | |
| P4 | 0 | 6 | 5 | 6 | 0 | 0 | 1 | 4 | 0 | 6 | 4 | 2 | | | | |

CSDN @风云诀

再对系统进行安全性分析：

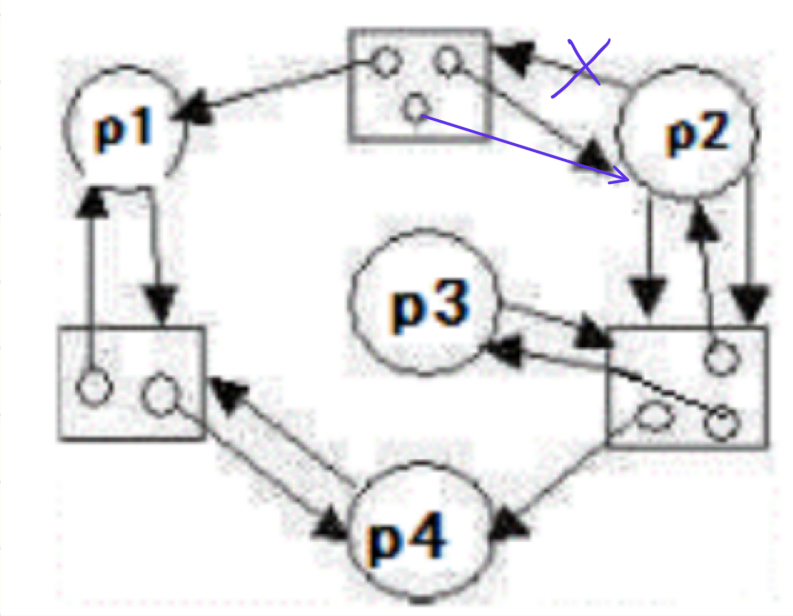
| 进程 | Work | | | | Need | | | | Allocation | | | | Available | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | A | B | C | D | A | B | C | D | A | B | C | D |
| P0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 2 | 1 | 1 | 1 | 2 |
| P2 | 1 | 1 | 1 | 2 | 1 | 0 | 0 | 2 | 1 | 3 | 5 | 4 | 2 | 4 | 6 | 6 |
| P3 | 2 | 4 | 6 | 6 | 0 | 0 | 2 | 0 | 0 | 6 | 3 | 2 | 2 | 10 | 9 | 8 |
| P1 | 2 | 10 | 9 | 8 | 0 | 7 | 5 | 0 | 1 | 0 | 0 | 0 | 3 | 10 | 9 | 8 |
| P4 | 3 | 10 | 9 | 8 | 0 | 6 | 4 | 2 | 0 | 0 | 1 | 4 | 3 | 10 | 10 | 12 |

CSDN @风云诀

因为存在一个安全序列<P0、P2、P3、P1、P4>，所以系统仍然处于安全状态

进程 P1 这个请求应该马上被满足

4.化简下面的资源分配图，并判断系统中是否存在死锁。(图请见附件)




无临界资源，已发生死锁